Digital circuits on FPGA using VHDL & Nexys ARTIX-7 Training Board

Alpana Sirohi (IISER-Pune)
Supervisor : Seema Sharma

Many thanks to Raghunandan Shukla & Shashi Dugad (TIFR) for all the help

Outline

- Learning VHDL(**VHSIC Hardware description language**) to program FPGA
  - NEXYS ARTIX -7 training board

- Exercises summarised in these slides:
  - Adders (1-bit, 2-bit, 8-bit)
  - Latches
  - Flip Flops

- **For each exercise, I have included these snapshots in the following slides:**
  - VHDL code
  - Resistor Transfer Level (RTL) schematic representation
  - Implementation of source code (VHDL code corresponding to digital circuit) in FPGA
  - Test-bench code
  - Output of simulation using test-bench

Exercise 1 : Adders (for 1-bit)

- 1- bit adder overview
- 1-bit adder first try (**not efficient one**)
- 1-bit adder (by using **intermediate signals** in VHDL).
- 1-bit adder (using "**std_logic_vector**" to define 'n' number of inputs)
    - **Test-bench** code and **simulated output** for 1-bit adder.
    - Schematic representation of **input assignment** in FPGA after implementation.

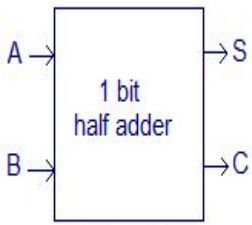
`

Half Adder

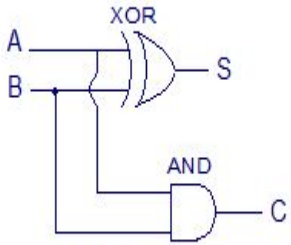
- It is a combinational digital circuit that is used for adding two numbers.
- A typical adder circuit produces a sum bit and a carry bit as the output.
- The half adder can add only two input bits and has nothing to do with the carry if there is any in the input.
- The binary addition process is not complete and that's why it is called a half adder.

Inputs		Outputs	
A	B	S	C
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Truth table



Schematic



Realization

http://www.circuitstoday.com/half-adder

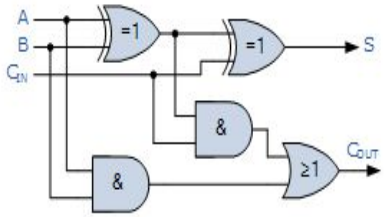
1-bit Full Adder

VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiatin
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity full_adder is
    Port ( a : in STD_LOGIC;
           b : in STD_LOGIC;
           cin : in STD_LOGIC;
           sum : out STD_LOGIC;
           cout : out STD_LOGIC);
end full_adder;

architecture Behavioral of full_adder is

begin
sum <= (a XOR b) XOR cin;
cout <= (a AND b) OR (cin AND (a XOR b));

end Behavioral;
```

Digital circuit and truth table for 1 bit adder

Symbol

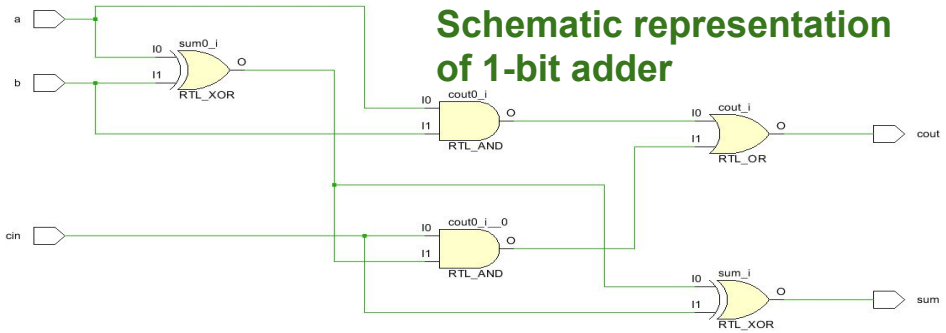


Truth Table

C-in	B	A	Sum	C-out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

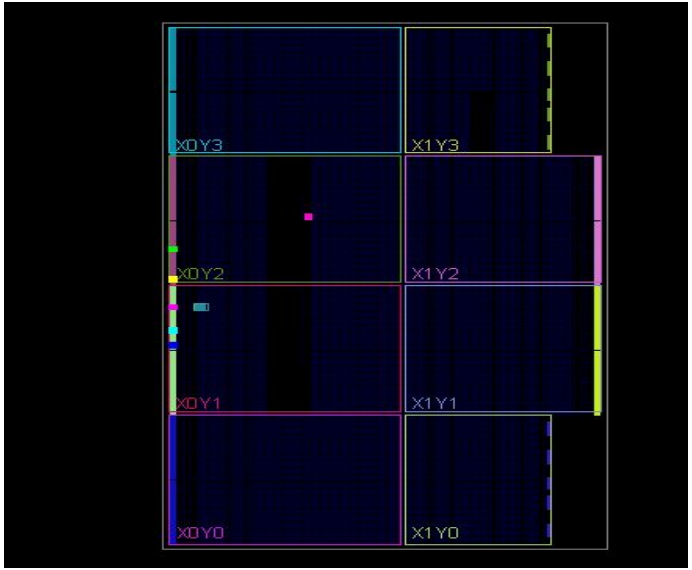
1-bit Full Adder : using intermediate signals

```
entity Full_Adder_0 is
    Port ( a : in STD_LOGIC;
           b : in STD_LOGIC;
           Cin : in STD_LOGIC;
           sum : out STD_LOGIC;
           Cout : out STD_LOGIC);
end Full_Adder_0;

architecture Behavioral of Full_Adder_0 is
    signal a0, b0,s0 : STD_LOGIC; -- intermediate states
begin
s0 <= a xor b;
sum <= s0 xor Cin ;
a0 <= a and b;
b0 <= Cin and s0;
Cout <= a0 or b0 ;

end Behavioral;
```



Schematic representation of 1-bit adder

Implemented design in FPGA



Test bench code and simulation output for 1-bit adder

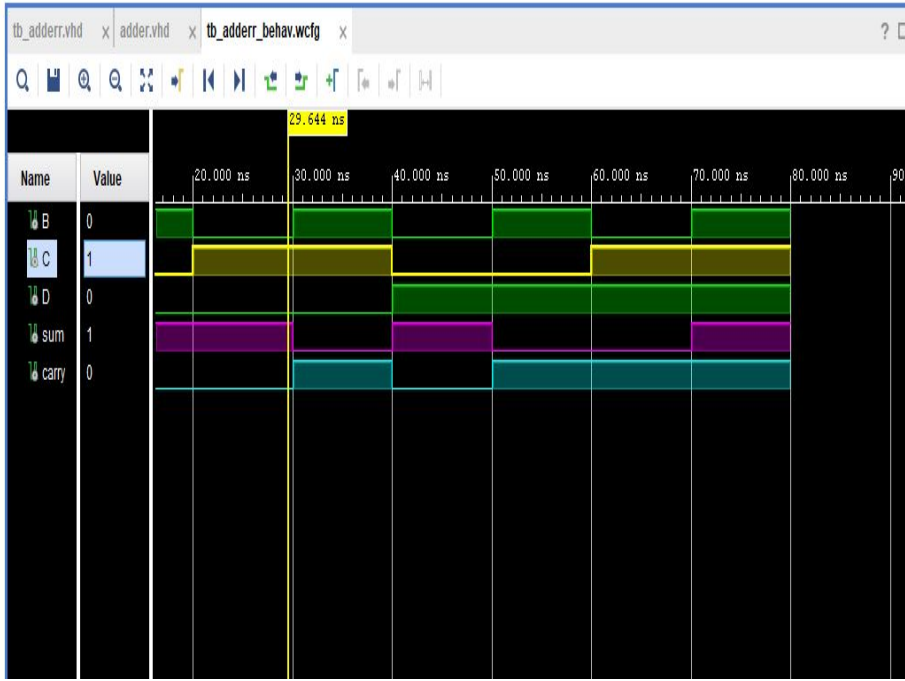
Source code

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity adder is
Port (
          a : in STD_LOGIC_VECTOR (2 downto 0);
        --  b : in STD_LOGIC;
         --cin : in STD_LOGIC;
          sum : out STD_LOGIC_VECTOR (1 downto 0));
end adder;

architecture Behavioral of adder is
begin
--sum(0) <= a(0) and a(1);
sum(0) <= (a(0) XOR a(1)) XOR a(2);
sum(1) <= (a(0) AND a(1)) OR (a(2) AND (a(0) XOR a(1)));
```

Test bench code

```vhdl
--use UNISIM.VComponents.all;
entity tb_adderr is
--  Port ( );
end tb_adderr;
architecture Behavioral of tb_adderr is

component adder is
    Port (
        a : in STD_LOGIC_vector (2 downto 0);

        sum: out STD_LOGIC_vector (1 downto 0)
    );
end component adder;

   -- signal a : STD_LOGIC_vector (2 downto 0) := "000";
    signal B : STD_LOGIC := '0';
   signal C : STD_LOGIC := '0';
   signal D : STD_LOGIC := '0';
    signal sum : STD_LOGIC;
    signal carry : STD_LOGIC;

begin

    UUT : adder
    Port map(
         a(0)=> B,
         a(1) => C,
         a(2) => D,
        --C => sum(0),
       sum(1) => carry,
       sum(0) => sum

    );

    B <= not B after 10ns ;
    C <= not C after 20ns ;
    D <= not  D after 40ns ;
    --D <= not D after 80 ns;

end Behavioral;
```

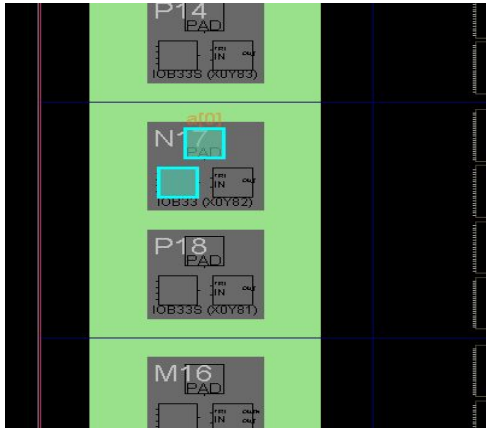
Simulation output



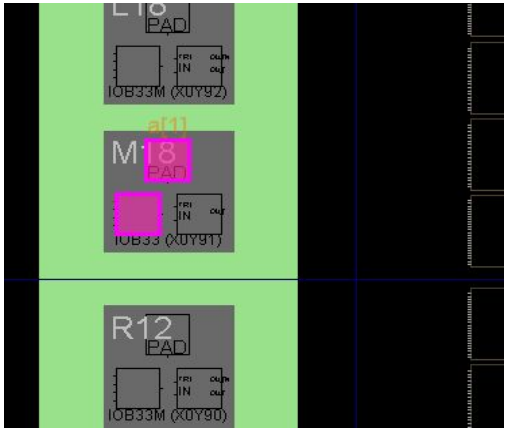
Schematic representation of 1-bit adder in FPGA after implementation

Zooming in inputs after implementation in FPGA

a(0)



a(1)

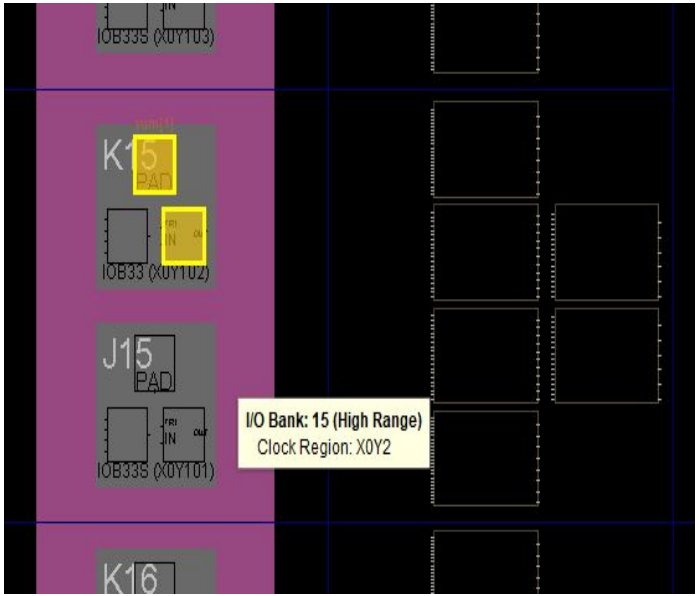


a(2)

Schematic representation of output 1-bit adder in FPGA after implementation

sum[0]

sum[1]



Exercise 2 : **Adders (2-bit and 8-bit)**

- 2-bit Ripple Carry adder (**sequentially** generating the carry)
  - Overview of 2-bit adder
  - VHDL code, Schematic representation
  - Test bench and simulated output

- 8-bit Ripple Carry adder (**sequentially** generating the carry)
  - Overview of 8-bit RIpple Carry adder
  - VHDL code, Schematic representation
  - Test bench and simulated output

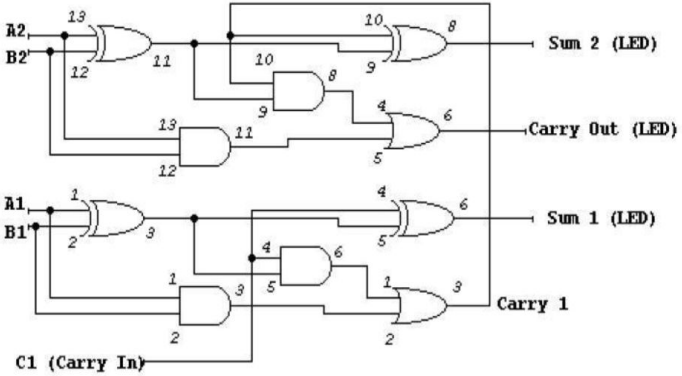
- 8-bit Carry Look Ahead adder (generating the carries in **parallel** to avoid gate delays)
  - Overview of 8-bit Carry Look Ahead adder
  - VHDL code, Schematic representation
  - Test bench and simulated output

2-bit Adder

- 2-bit adder can be made using two full adder.
- Carry output for first adder will work as an input carry for second adder.

Truth table

INPUTS					OUTPUTS		
A0	A1	B0	B1	Ci	S0	S1	Co
0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	0	0	1	0
0	1	0	1	0	0	0	1
0	1	0	0	1	1	1	0
1	0	1	0	1	1	1	0
1	1	1	1	1	1	1	1



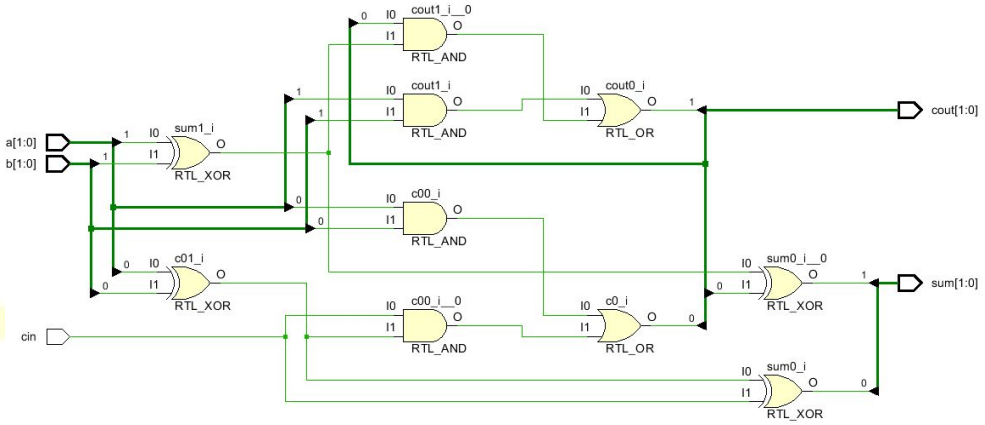
Circuit diagram

2-bit Ripple Carry Adder

VHDL Code

```
entity ripple_2bit_adder is
    Port ( a0 : in STD_LOGIC;
           b0 : in STD_LOGIC;
           a1 : in STD_LOGIC;
           b1 : in STD_LOGIC;
           cin : in STD_LOGIC;
           sum0 : out STD_LOGIC;
           cout0 : out STD_LOGIC;
            sum1 : out STD_LOGIC;
           cout1 : out STD_LOGIC);
end ripple_2bit_adder;

architecture Behavioral of ripple_2bit_adder is
    signal c0 :STD_LOGIC;
begin
sum0 <= a0 xor b0 xor cin;
c0 <=  (a0 AND b0) OR (cin AND (a0 XOR b0));
sum1 <= a1 xor b1 xor c0;
cout0 <= c0;
cout1 <=  (a1 AND b1) OR (c0 AND (a1 XOR b1));

end Behavioral;
```

RTL schematic representation of VHDL code

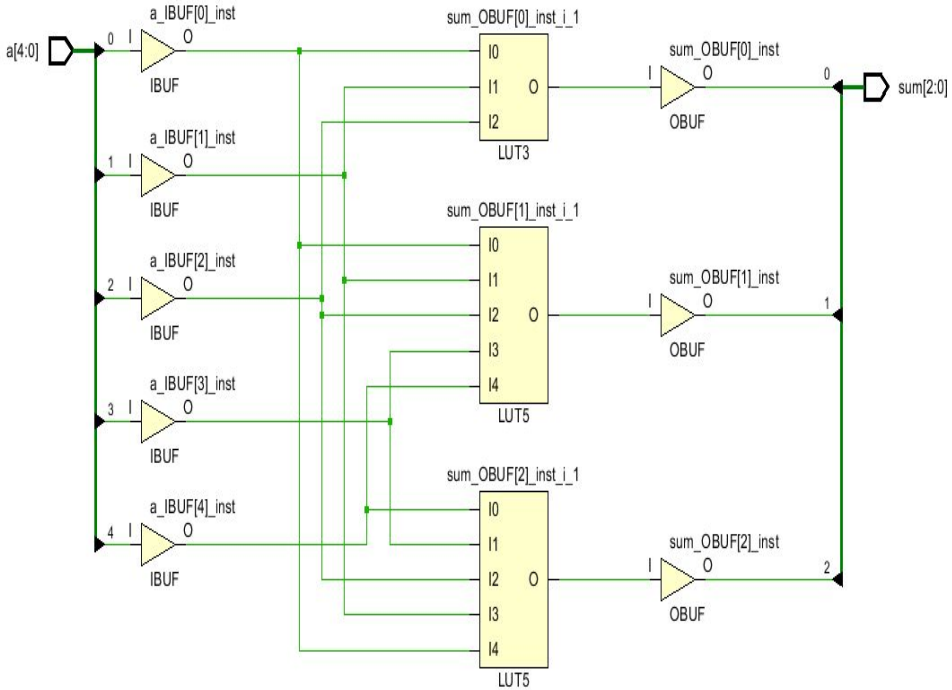


2-bit adder VHDL code using std_logic_vector

VHDL Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instant
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Full_2bit_adder is
    Port ( a : in STD_LOGIC_VECTOR (4 downto 0);
           sum : out STD_LOGIC_VECTOR (2 downto 0));
end Full_2bit_adder;

architecture Behavioral of Full_2bit_adder is
signal c0 : STD_LOGIC;
begin
sum(0) <= (a(1) XOR a(2)) XOR a(0);
c0 <= (a(1) AND a(2)) OR (a(0) AND (a(1) XOR a(2)));
sum(1) <= (a(3) XOR a(4)) XOR c0;
sum(2) <= (a(3) AND a(4)) OR (c0 AND (a(3) XOR a(4)));


end Behavioral;
```

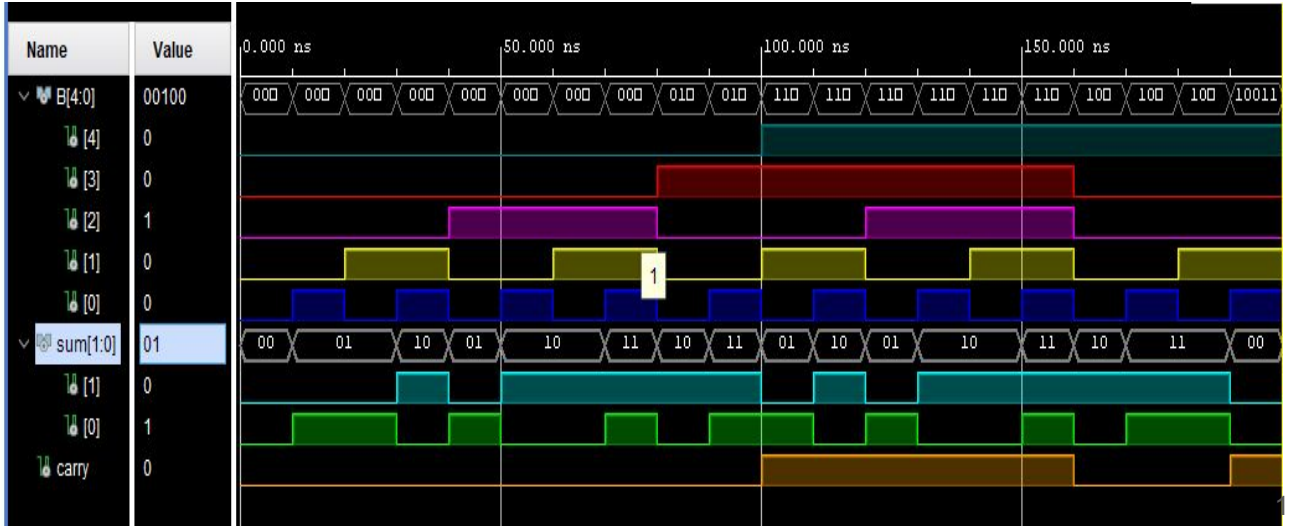
RTL Schematic representation after synthesizing the VHDL code



Test bench code and simulation output for 2-bit adder

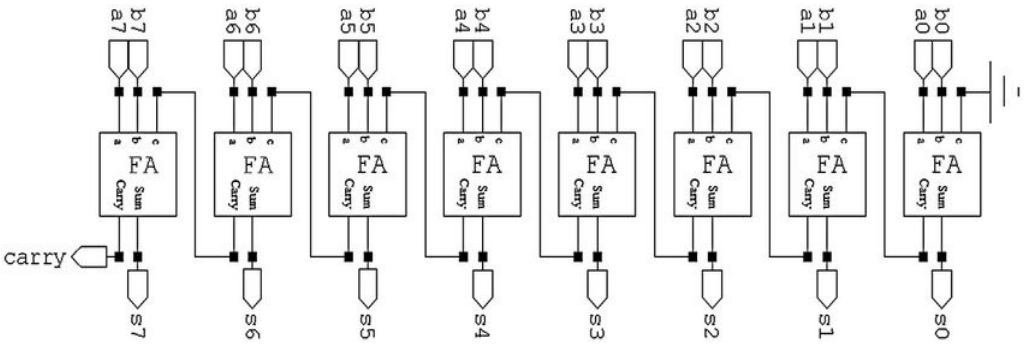
```
entity tb_Full_2bit_adder is
--  Port ( );
end tb_Full_2bit_adder;
architecture Behavioral of tb_Full_2bit_adder is
component  Full_2bit_adder is
    Port (
        a : in STD_LOGIC_vector (4 downto 0);

        sum: out STD_LOGIC_vector (2 downto 0)
    );
end component Full_2bit_adder;

   -- signal a : STD_LOGIC_vector (2 downto 0) := "000";
    signal B : STD_LOGIC_VECTOR (4 downto 0) := "00000";
   --signal C : STD_LOGIC := '0';
   --signal D : STD_LOGIC := '0';
    signal sum : STD_LOGIC_VECTOR(1 downto 0);
    signal carry : STD_LOGIC;

begin

    UUT : Full_2bit_adder
    Port map(
         a(0)=> B(0),
         a(1) => B(1),
         a(2) => B(2),
        a(3)=> B(3),
        a(4) => B(4),
       sum(1) => sum(1),
       sum(0) => sum(0),
       sum(2)=> carry

    );

    B(0) <= not B(0) after 10ns ;
    B(1) <= not B(1) after 20ns ;
    B(2) <= not  B(2) after 40ns ;
    B(3)<= not B(3) after 80 ns;
     B(4)<= not B(4) after 100ns;

end Behavioral;
```

1. Test bench code
2. Simulation output



8-bit adder (Ripple carry adder)

- It is built as a cascade from eight 1-bit full-adders.
- Each stage of the adder has to wait until the previous stage has calculated and propagates its carry output signal.
- The total delay of a ripple-carry adder is proportional to the number of bits.
- Faster adders are often required for bit widths of 16 or greater.



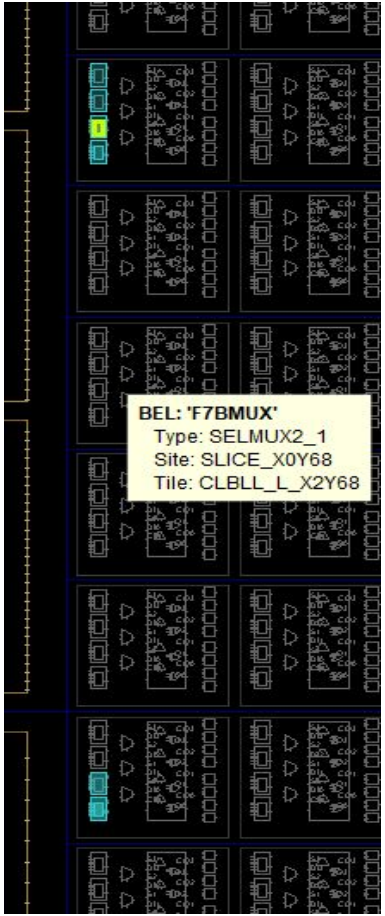
https://tams.informatik.uni-hamburg.de/applets/hades/webdemos/20-arithmetic/10-adders/ripple.html

8-bit ripple carry adder

VHDL code

```
entity adder_8bit_ripple is
    Port ( in_puts : in STD_LOGIC_VECTOR (16 downto 0);
           out_puts : out STD_LOGIC_VECTOR (8 downto 0));
end adder_8bit_ripple;

architecture Behavioral of adder_8bit_ripple is
signal carry : STD_LOGIC_VECTOR (6 downto 0);
begin

carry(0) <= (in_puts(1) AND in_puts(2)) OR (in_puts(0)  AND (in_puts(1) XOR in_puts(2)));
carry(1) <= (in_puts(3) AND in_puts(4)) OR (carry(0) AND (in_puts(3) XOR in_puts(4)));
carry(2) <= (in_puts(5) AND in_puts(6)) OR (carry(1) AND (in_puts(5) XOR in_puts(6))) ;
carry(3) <= (in_puts(7) AND in_puts(8)) OR (carry(2) AND (in_puts(7) XOR in_puts(8))) ;
carry(4) <= (in_puts(9) AND in_puts(10)) OR (carry(3) AND (in_puts(9) XOR in_puts(10))) ;
carry(5) <= (in_puts(11) AND in_puts(12)) OR (carry(4) AND (in_puts(11) XOR in_puts(12))) ;
carry(6) <= (in_puts(13) AND in_puts(14)) OR (carry(5) AND (in_puts(13) XOR in_puts(14))) ;
out_puts(1) <= (in_puts(3) XOR in_puts(4)) XOR carry(0);
out_puts(0) <= (in_puts(1) XOR in_puts(2)) XOR in_puts(0);
out_puts(2)<= (in_puts(5) XOR in_puts(6)) XOR carry(1);
out_puts(3) <= (in_puts(7) XOR in_puts(8)) XOR carry(2);
out_puts(4) <= (in_puts(9) XOR in_puts(10)) XOR carry(3);
out_puts(5) <= (in_puts(11) XOR in_puts(12)) XOR carry(4);
out_puts(6) <= (in_puts(13) XOR in_puts(14)) XOR carry(5);
out_puts(7) <= (in_puts(15) XOR in_puts(16)) XOR carry(6);
out_puts(8) <= (in_puts(15) AND in_puts(16)) OR (carry(6) AND (in_puts(15) XOR in_puts(16)));
end Behavioral;
```

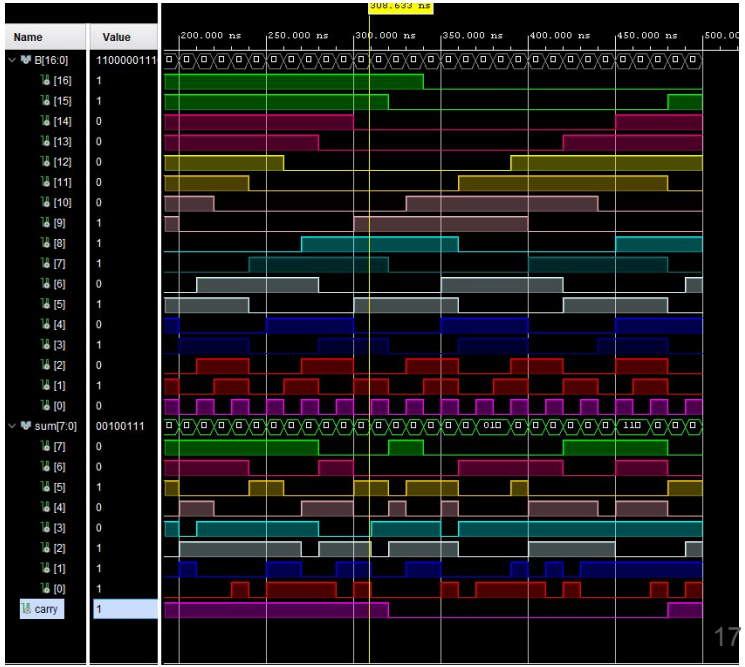
implemented design in FPGA



Test bench code and simulation output for 8-bit adder

```
architecture Behavioral of tb_adder_8bit_ripple is
component  adder_8bit_ripple is
    Port (
        in_puts : in STD_LOGIC_vector (16 downto 0);

        out_puts : out STD_LOGIC_vector (8 downto 0)
    );
end component adder_8bit_ripple;

   -- signal a : STD_LOGIC_vector (2 downto 0) := "000";
    signal B : STD_LOGIC_VECTOR (16 downto 0) := "00000000000000000";
   --signal C : STD_LOGIC := '0';
   --signal D : STD_LOGIC := '0';
    signal sum : STD_LOGIC_VECTOR(7 downto 0);
    signal carry : STD_LOGIC;

begin

    UUT : adder_8bit_ripple
    Port map(
         in_puts(0)=> B(0),
         in_puts(1) => B(1),
         in_puts(2) => B(2),
        in_puts(3)=> B(3),
        in_puts(4) => B(4),
         in_puts(5)=> B(5),
         in_puts(6) => B(6),
         in_puts(7) => B(7),
        in_puts(8)=> B(8),
        in_puts(9) => B(9),
         in_puts(10)=> B(10),
         in_puts(11) => B(11),
         in_puts(12) => B(12),
        in_puts(13)=> B(13),
        in_puts(14) => B(14),
         in_puts(15)=> B(15),
         in_puts(16) => B(16),

       out_puts(1) => sum(1),
       out_puts(0) => sum(0),
       out_puts(1) => sum(1),
       out_puts(0) => sum(0),
        out_puts(2) => sum(2),
       out_puts(3) => sum(3),
        out_puts(4) => sum(4),
       out_puts(5) => sum(5),
        out_puts(6) => sum(6),
       out_puts(7) => sum(7),
       out_puts(8)=> carry

    );

    B(0) <= not B(0) after 10ns ;
    B(1) <= not B(1) after 20ns ;
    B(2) <= not  B(2) after 30ns ;
    B(3)<= not B(3) after 40ns;
    B(4)<= not B(4) after 50ns;

    B(5) <= not B(5) after 60ns ;
    B(6) <= not B(6) after 70ns ;
    B(7) <= not  B(7) after 80ns ;
    B(8)<= not B(8) after 90ns;
     B(9)<= not B(9) after 100ns;

    B(10) <= not B(10) after 110ns ;
    B(11) <= not B(11) after 120ns ;
    B(12) <= not  B(12) after 130ns ;
    B(13)<= not B(13) after 140ns;
     B(14)<= not B(14) after 150ns;
     B(15)<= not B(15) after 160ns;
     B(16) <= not B(16) after 170ns;

end Behavioral;
```

1. Test bench code
2. Simulation output



Carry Look Ahead adder

- To avoid gate delays we use CLA adder.
- The carry input at any stage of the adder is independent of the carry bits generated at the independent stages.
- Output of any stage i, can be written as,

  - $S_i = P_i \oplus G_i$.
  - $C_{i+1} = C_i.P_i + G_i$.

hence,

$C1 = C0.P0+G0.$

$C2 = C1.P1+G1 = (C0.P0+G0).P1+G1.$

$C3 = C2.P2+G2 = (C1.P1+G1).P2+G2.$

$C4 = C3.P3+G3 = C0.P0.P1.P2.P3 + P3.P2.P1.G0 + P3.P2.G1 + G2.P3 + G3.$

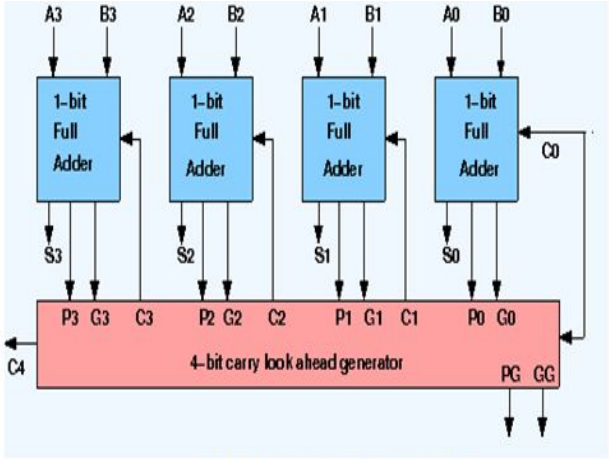


Fig3: Carry Look ahead Adders

Drawbacks

- It provides high speed but the complexity of the circuit is also increased thereby increasing the area occupied by the circuit.
- This adder is costlier compared to other adders

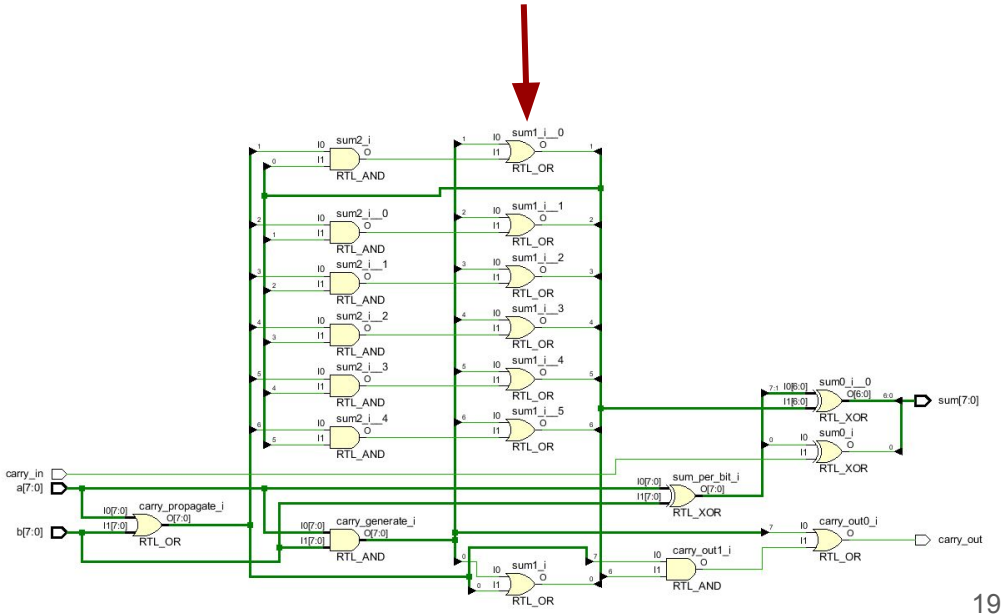
https://www.elprocus.com/carry-look-ahead-adder/

8-bit carry look (parallel) ahead adder

```vhdl
entity carry_look_ahead_adder is
    Port ( a : in STD_LOGIC_VECTOR (7 downto 0);
           b : in STD_LOGIC_VECTOR (7 downto 0);
           carry_in : in STD_LOGIC;
           sum : out STD_LOGIC_VECTOR (7 downto 0);
           carry_out : out STD_LOGIC);
end carry_look_ahead_adder;

architecture Behavioral of carry_look_ahead_adder is
signal carry_generate : STD_LOGIC_VECTOR(7 downto 0);
signal carry_propagate : STD_LOGIC_VECTOR(7 downto 0);
signal carry_internal : STD_LOGIC_VECTOR(7 downto 1);
signal sum_per_bit : STD_LOGIC_VECTOR(7 downto 0);
begin
sum_per_bit <= a xor b;
carry_generate <= a and b;
carry_propagate <= a or b;

process (carry_generate,carry_propagate,carry_internal)
begin
    carry_internal(1) <= carry_generate(0) or carry_propagate(0);

    main :  FOR i IN 1 TO 6 LOOP
            carry_internal(i+1) <= carry_generate(i) OR (carry_propagate(i) AND carry_internal(i));
            END LOOP;
   carry_out <= carry_generate(7) OR (carry_propagate(7) AND carry_internal(7));
   END PROCESS;
   sum(0) <= sum_per_bit(0) XOR carry_in;
   sum(7 DOWNTO 1) <= sum_per_bit(7 DOWNTO 1) XOR carry_internal(7 DOWNTO 1);
end Behavioral;
```

Source vhdl code for generating carry parallely

RTL schematic representation of CLA adder



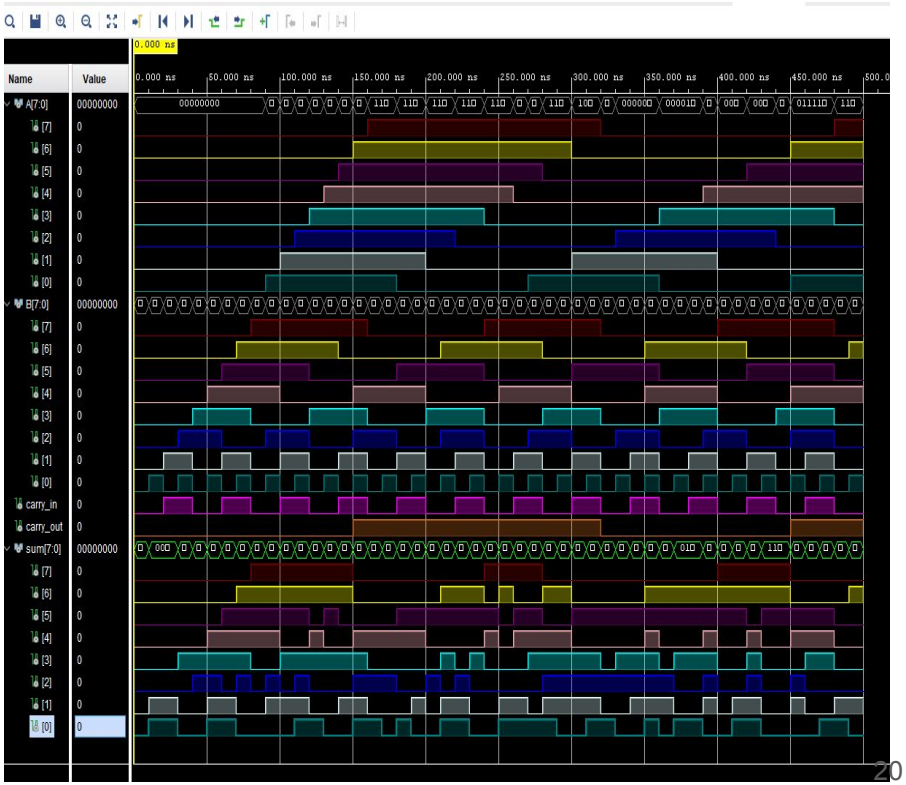
Test bench code and simulation output for 8-bit adder : Carry Look Ahead adder (CLA)

Test bench code for CLA adder

```
        a: in STD_LOGIC_vector (7 downto 0);
        b : in STD_LOGIC_VECTOR (7 downto 0);
        carry_in : in STD_LOGIC;
        carry_out : out std_logic;
        sum : out STD_LOGIC_vector (7 downto 0)
    );
end component carry_look_ahead_adder;

   signal A : STD_LOGIC_vector (7 downto 0) := "000000
    signal B : STD_LOGIC_VECTOR (7 downto 0) := "00000
   signal carry_in : STD_LOGIC := '0';
   signal carry_out : STD_LOGIC ;
    signal sum : STD_LOGIC_VECTOR(7 downto 0);
  --  signal carry : STD_LOGIC;

begin

    UUT : carry_look_ahead_adder
    Port map(
a(0)=> B(0),
a(1) => B(1),
a(2) => B(2),

a(5)=> B(5),
a(6) => B(6),
a(7) => B(7),
a(3)=> B(3),
a(4) => B(4),


b(0)=> A(0),
b(1) => A(1),
b(3)=> A(3),
b(4) => A(4),
b(7)=> A(7),
b(6) => A(6),
b(2)=> A(2),
b(5) => A(5),

 carry_in => carry_in,
       sum(1) => sum(1),
      sum(0) => sum(0),
```

```
 carry_in => carry_in,
       sum(1) => sum(1),
      sum(0) => sum(0),
       sum(2) => sum(2),
      sum(3) => sum(3),
       sum(4) => sum(4),
      sum(5) => sum(5),
       sum(6) => sum(6),
      sum(7) => sum(7),
carry_out => carry_out

    );

    B(0) <= not B(0) after 10ns ;
    B(1) <= not B(1) after 20ns ;
    B(2) <= not  B(2) after 30ns ;
    B(3)<= not B(3) after 40ns;
    B(4)<= not B(4) after 50ns;

    B(5) <= not B(5) after 60ns ;
    B(6) <= not B(6) after 70ns ;
    B(7) <= not  B(7) after 80ns ;

    A(0)<= not A(0) after 90ns;
     A(1)<= not A(1) after 100ns;

    A(2) <= not A(2) after 110ns ;
    A(3) <= not A(3) after 120ns ;
    A(4) <= not  A(4) after 130ns ;
    A(5)<= not A(5) after 140ns;
     A(6)<= not A(6) after 150ns;
     A(7)<= not A(7) after 160ns;
     --B(16) <= not B(16) after 170ns;
 carry_in <= not carry_in after 20ns;

end Behavioral;
```

Simulation output of carry look ahead adder



Exercise 3 : **simple latches (without any clock)**

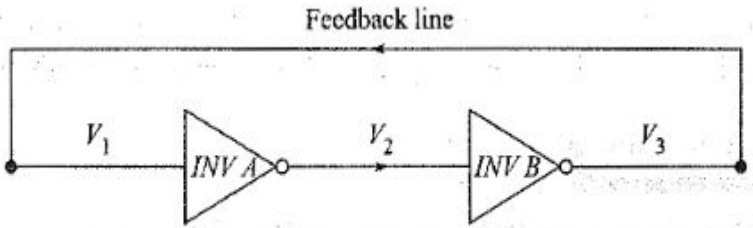
- **Flip flops and their importance**

- **D Latch**
  - Overview of D- latch
  - VHDL code
  - Schematic representation
  - Test bench code and simulated output

- **SR latch**
  - Overview of SR- latch
  - VHDL code
  - Schematic representation
  - Test bench code and simulated output

Flip Flops and their importance

- The outputs of the digital circuits considered previously are dependent entirely on their inputs.
- But there are requirements for a digital device or circuit whose output will remain unchanged with a change in inputs.
- Such a device could be used to store a binary number.
- A flip flop is a bistable electronic circuit that has two stable states.
- The flip-flop also has memory since its output will remain as set until something is done to change it.
- Latches or Flip flops are the basic building block of memory.



(a) Bistable circuit

http://soaneemrana.com/onewebmedia/DIGITAL%20PRINCIPLES%20AND%20APPLICATION%20BY%20LEACH%20&%20MALVINO.pdf

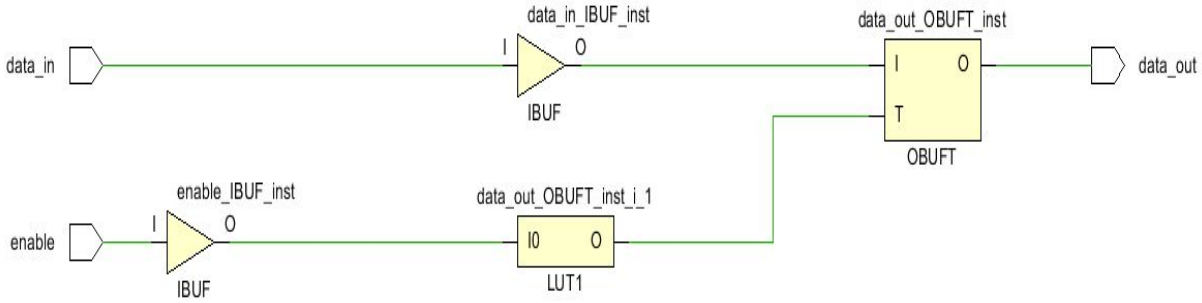
Simple or D- latch

Simple or D- latch

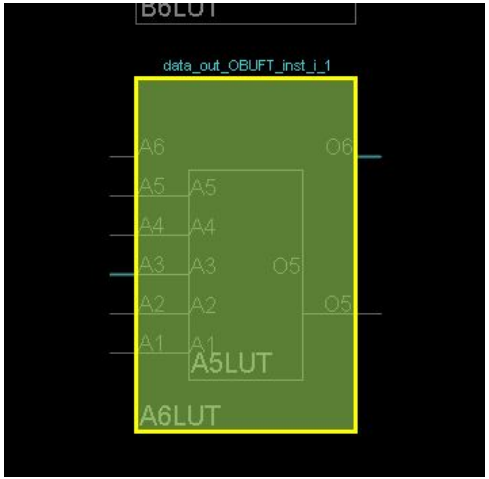
VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if us
-- arithmetic functions with Signed or Unsigned valu
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if in
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity simple_latch is
 Port (
data_in : in STD_LOGIC;
enable : in STD_LOGIC;
--enable : in STD_LOGIC;
data_out : out STD_LOGIC );

end simple_latch;

architecture Behavioral of simple_latch is
-- a : STD_LOGIC;
begin
--data_out <= a;
process( enable, data_in)
--variable tmp : std_logic;
begin
if ( enable = '1') then
--if(CLOCK='1' and CLOCK'EVENT) then
data_out <= data_in;
else
 data_out <= 'Z';
--end if;
end if;
end process;
--data_out <= a;
end Behavioral;
```

Schematic representation of source code for D latch



Implemented design in fpga

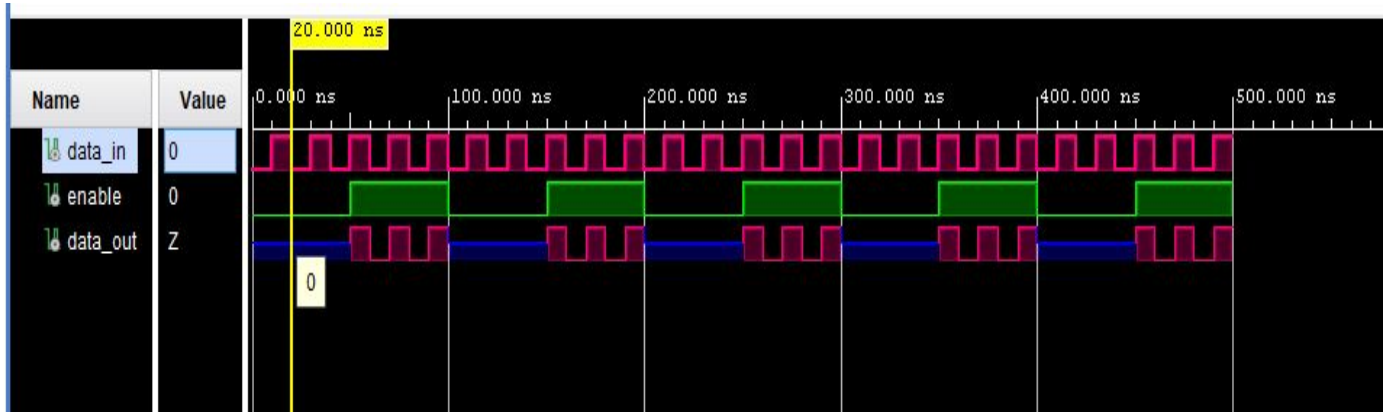


Test bench code and simulation output for Simple latch(D- latch)

```
entity tb_simple_latch is
--  Port ( );
end tb_simple_latch;

architecture Behavioral of tb_simple_latch

component simple_latch is
    Port (
        data_in : in STD_LOGIC;
     --   R : in STD_LOGIC;
        enable : in STD_LOGIC;
        data_out : out STD_LOGIC
        --QBAR : out STD_LOGIC
    );
end component simple_latch;

    signal data_in : STD_LOGIC := '0';
    --signal R : STD_LOGIC := '0';
    signal enable : STD_LOGIC := '0';
   -- signal Q : STD_LOGIC ;
    signal data_out : STD_LOGIC;


begin

    UUT : simple_latch
    Port map(
        data_in => data_in,
       -- R => R,
        enable => enable,
        data_out => data_out
        --QBAR => QBAR
    );
     data_in <= not data_in after 10 ns;
    --R <= not R after 20 ns;
    enable <= not enable after 50 ns;
   -- D <= not D after 80 ns;

end Behavioral;
```

Test bench code

Simulated output



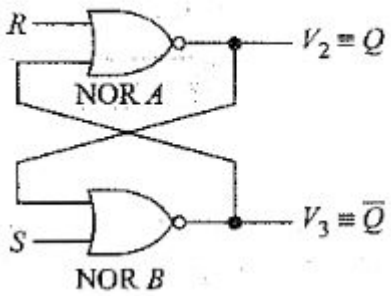
SR Latch

- When R = 0 and S = 0. Since a O at the input of a NOR gate has no effect on its output, the flip-flop simply remains in its present state; that is, Q remains unchanged.

- When R = 0 and S = 1, then output of NOR gate B will be low. Thus a 1 at the S input is said to SET the flip-flop, and it switches to the stable state where
  Q = 1.

- R = 1 and S = 1, is forbidden, as it forces the outputs of both NOR gates to the low state. In other words, both Q = 0 and Q = 0 at the same time.

Digital circuit



Truth table

R	S	Q	Action
0	0	Last state	No change
0	1	1	SET
1	0	0	RESET
1	1	?	Forbidden

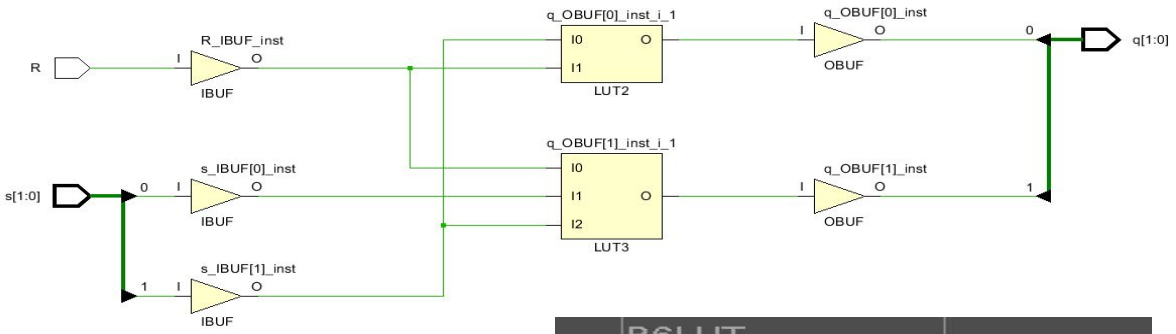
http://soaneemrana.com/onewebmedia/DIGITAL%20PRINCIPLES%20AND%20APPLICATION%20BY%20LEACH%20&%20MALVINO.pdf

SR Latch

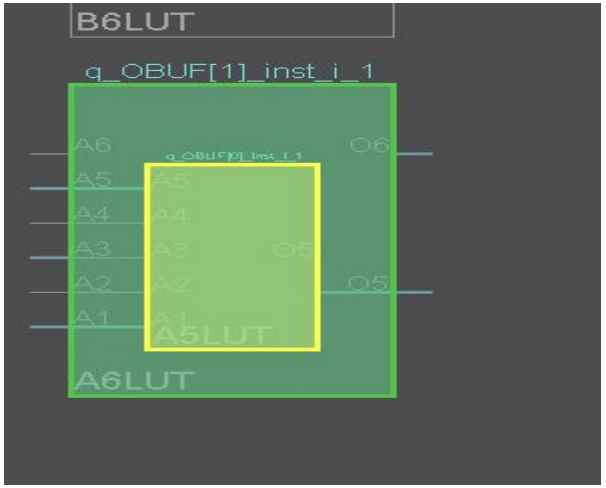
VHDL code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if
-- arithmetic functions with Signed or Unsigned v
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity rs_latch is
    Port ( s : in STD_LOGIC_VECTOR (1 downto 0);
           q : out STD_LOGIC_VECTOR (1 downto 0);
           R : in STD_LOGIC );
end rs_latch;

architecture Behavioral of rs_latch is
signal p : STD_LOGIC_VECTOR (1 downto 0) := "10";
begin
latch : process(s,p) is
begin
    if ( R = '1') then
    q <= p;
    elsif ( s = "00") then
     q <= p ;
    elsif( s = "01" or s = "10") then
     q <= not s;
   else
   q <= "11";
  -- end if;
   end if;
   end process latch;


end Behavioral;
```

Schematic RTL design for source code



Implemented design in fpga

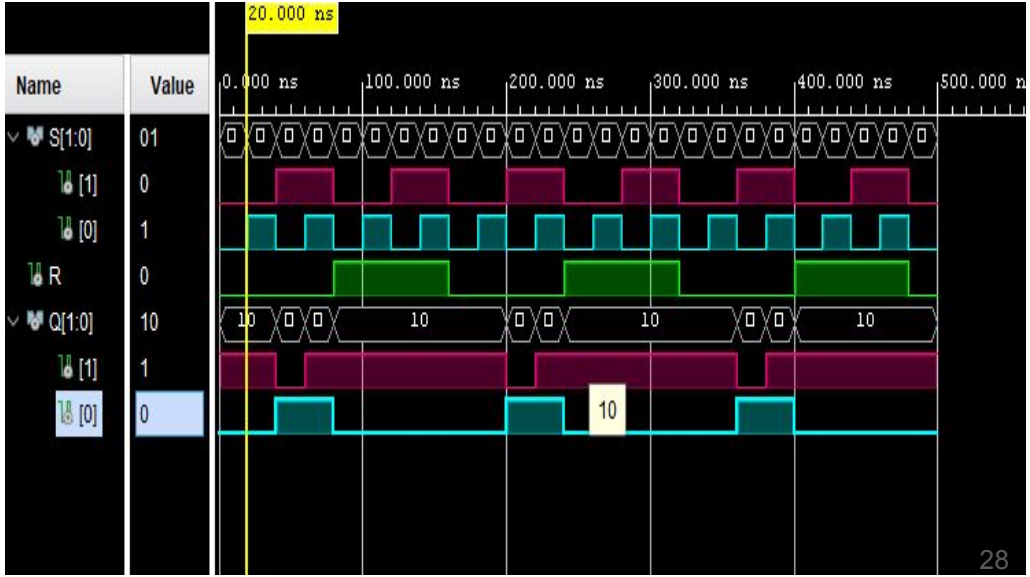


Test bench code and simulation output for SR latch

```
entity tb_rs_latch is
--  Port ( );
end tb_rs_latch;

architecture Behavioral of tb_rs_latch is

component rs_latch is
    Port (
        s : in STD_LOGIC_VECTOR(1 downto 0);
      R : in STD_LOGIC;
        --enable : in STD_LOGIC;
        Q : out STD_LOGIC_VECTOR (1 downto 0)
        --QBAR : out STD_LOGIC
    );
end component rs_latch;

    signal S : STD_LOGIC_VECTOR(1 downto 0) := "00"
   signal R : STD_LOGIC := '0';
    --signal CLOCK : STD_LOGIC := '0';
    signal Q : STD_LOGIC_VECTOR(1 downto 0) ;
    --signal QBAR : STD_LOGIC;


begin

    UUT : rs_latch
    Port map(
        S => S,
        R => R,
      --  CLOCK => CLOCK,
        Q => Q
       -- QBAR => QBAR
    );
     S(0) <= not S(0) after 20 ns;
     S(1) <= not S(1) after 40ns;
    R <= not R after 80 ns;
    --CLOCK <= not CLOCK after 40 ns;
```

Test bench code

Simulated output



Exercise 4 : **Flip Flops**

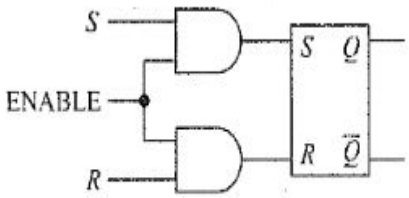
- **SR flip flops**
  - Overview of SR flip flops
  - VHDL code, Implemented and Schematic representation
  - Test bench and simulated output

- **D flip flops**
  - Overview of D flip flops
  - VHDL code, Implemented and Schematic representation
  - Test bench and simulated output

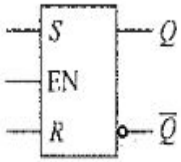
- **JK flip flops**
  - Overview of JK flip flops
  - VHDL code, Implemented and Schematic representation
  - Test bench and simulated output

SR flip-flop

- It is a flip-flop that can be enabled or disabled.
- When the ENABLE input is low, the latch is said to be disabled.
- When the ENABLE input is high, information at the R and S inputs will be transmitted directly to the outputs. The latch is said to be enabled.
- When the ENABLE input goes low, the output will retain the information that was present on the input when the high-to-low transition took place.
- The RS flip-flop has two data inputs, R and S. To store a high bit, you need a high S; to store a low bit, you need a high R.
- So we can hold the stored information for any desired period of time.



(a) Logic diagram



EN	S	R	Q_{n+1}
1	0	0	Q_n (no change)
1	0	1	0
1	1	0	1
1	1	1	? (Illegal)
0	X	X	Q_n (no change)

(b) IEEE symbol and truth table

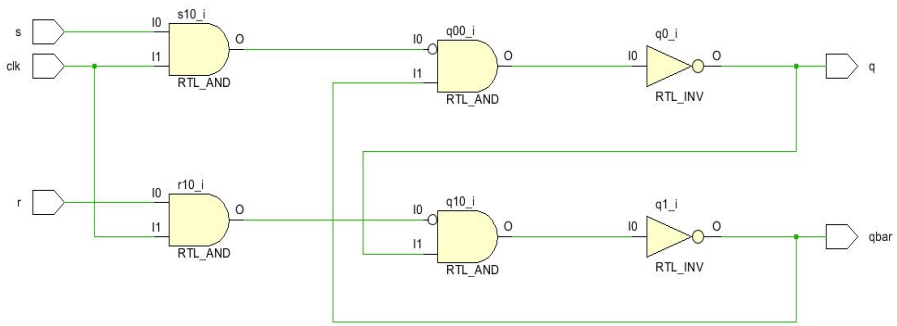
http://soaneemrana.com/onewebmedia/DIGITAL%20PRINCIPLES%20AND%20APPLICATION%20BY%20LEACH%20&%20MALVINO.pdf

SR flip flop

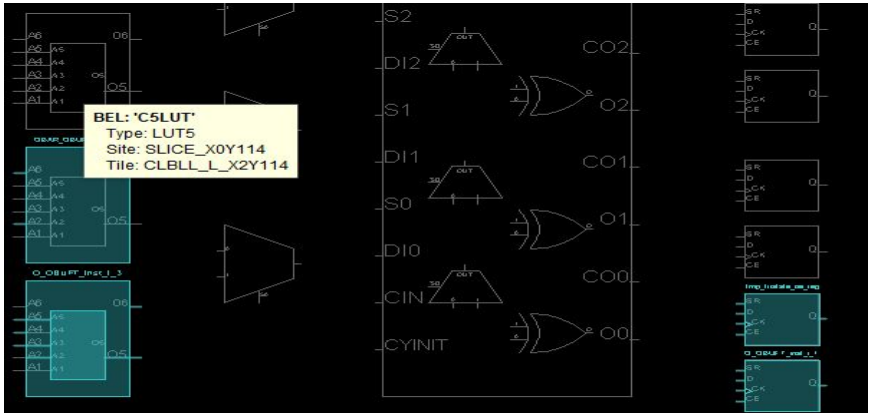
VHDL code

```
--library UNISIM;
--use UNISIM.VComponents.all;

entity rs_clocked_latch is
    Port ( s : in STD_LOGIC_VECTOR (1 downto 0);
           q : out STD_LOGIC_VECTOR (1 downto 0);
           reset : in STD_LOGIC;
           CLOCK : in STD_LOGIC);
end rs_clocked_latch;

architecture Behavioral of rs_clocked_latch is
 signal p : STD_LOGIC_VECTOR ( 1 downto 0) := "10";
begin
process (CLOCK)

                begin
                     if ( reset = '1') then
                     q <= "01";



                      elsif (CLOCK ='1' and CLOCK'EVENT ) then
                       if ( s = "00") then
                            q <= p ;
                      -- elsif( s = "11") then
                        -- p <='Z';
                       elsif( s = "01" or s = "10") then
                      q <= not s;
                      elsif( s ="11") then

                     q <= "ZZ";
                       end if;
                       --else
                       --q <= p;
                --end process ;
                end if;
                  --  end if;
                end process;



end Behavioral;
```

Schematic rtl design



Implemented on fpga

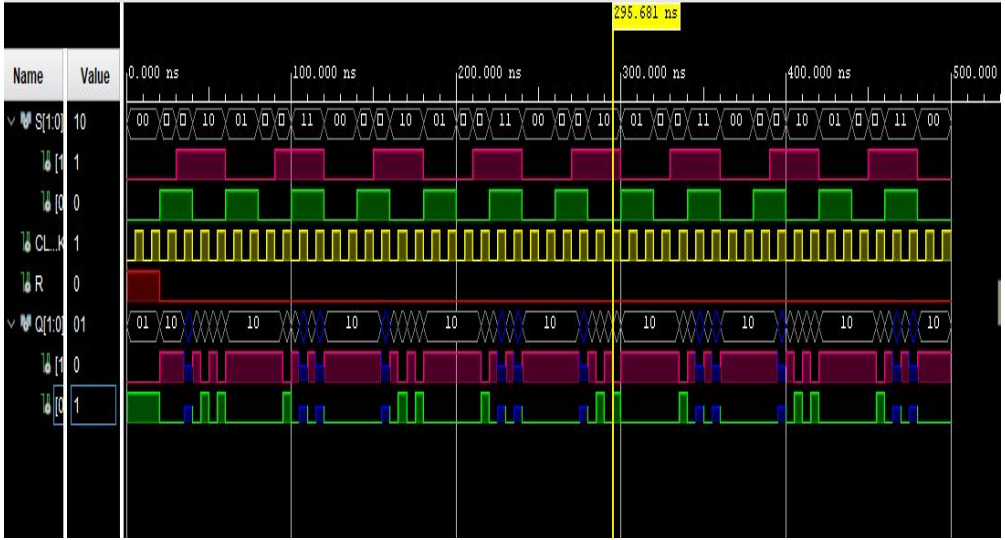


Test bench code and simulation output for SR flip-flop

Test bench code

```
    --signal QBAR : STD_LOGIC;
    constant clk_period : time := 10 ns;

begin

    UUT : rs_clocked_latch
    Port map(
        s => S,
        reset => R,
        CLOCK => CLOCK,
        q=> Q
        --QBAR => QBAR
    );
    clk_process :process
   begin
   clock <= '0';
   wait for clk_period/2;
   clock <= '1';
   wait for clk_period/2;
   end process;

     stim_proc: process
  begin

  R <= '1';
  wait for 20 ns;

  R <= '0';
   wait for 50 ns;

  R <= '0';
  wait;
  end process;

     S(0) <= not S(0) after 20 ns;
     S(1) <= not S(1) after 40ns;
     -- R <= not R after 3 ns;

   -- D <= not D after 80 ns;


end Behavioral;
```

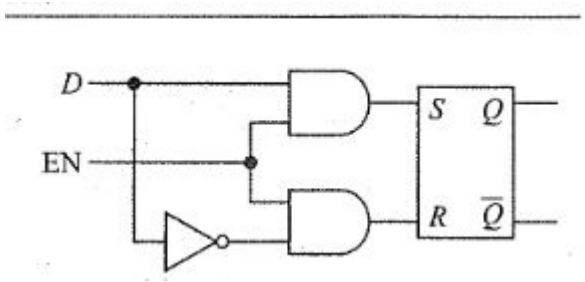
Simulated output



Simple Latch with clock(D flip-flop)

- Generation of two signals to drive a flip-flop is a disadvantage in many applications and the forbidden condition of both R and S high may occur accidently.
- To avoid that , we need D flip-flop, a circuit that needs only a single data input.
- This flip-flop is disabled when EN is low, but is transparent when EN is high.

Digital circuit



Truth table

EN	D	Q_{n+1}
0	X	Q_n (last state)
1	0	0
1	1	1

(b) Truth table

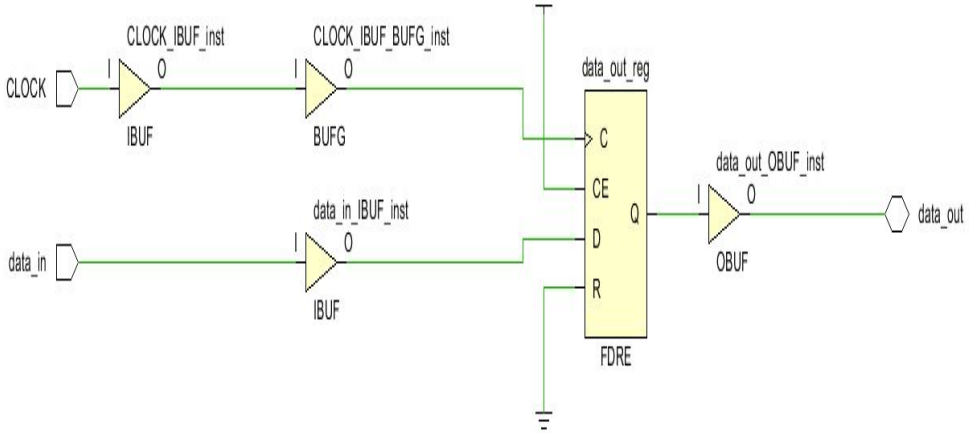
http://soaneemrana.com/onewebmedia/DIGITAL%20PRINCIPLES%20AND%20APPLICATION%20BY%20LEACH%20&%20MALVINO.pdf

Simple Latch with clock(D flip-flop)

VHDL code

```
--library UNISIM;
--use UNISIM.VComponents.all;

entity simple_latch is
 Port (
data_in : in STD_LOGIC;
--enable : in STD_LOGIC;
CLOCK : in STD_LOGIC;
data_out : inout STD_LOGIC );

end simple_latch;

architecture Behavioral of simple_latch is
-- a : STD_LOGIC;
begin
--data_out <= a;
process(  data_in, CLOCK)
--signal tmp : std_logic := '0';
begin

if(CLOCK='1' and CLOCK'EVENT) then
--if ( enable = '1') then
--data_out <= '0';
--elsif ( enable = '0') then
data_out <= data_in;
--else
--data_out <= 'Z';
end if;
--end if;
end process;
--data_out <= a;
end Behavioral;
```

RTL schematic representation of a latch

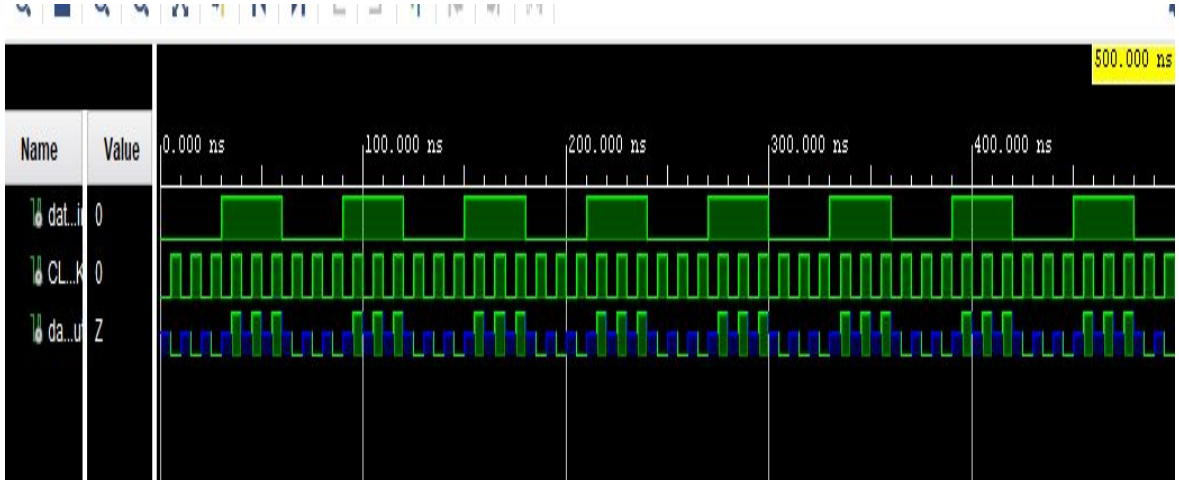


Test bench code and simulation output for D flip-flop

Test bench code

```
--library UNISIM;
--use UNISIM.VComponents.all;

entity tb_simple_latch is
--  Port ( );
end tb_simple_latch;

architecture Behavioral of tb_simple_latch i

component simple_latch is
    Port (
        data_in : in STD_LOGIC;
     --   R : in STD_LOGIC;
        CLOCK : in STD_LOGIC;
        data_out : out STD_LOGIC
        --QBAR : out STD_LOGIC
    );
end component simple_latch;

    signal data_in : STD_LOGIC := '0';
    --signal R : STD_LOGIC := '0';
    signal CLOCK : STD_LOGIC := '1';
   -- signal Q : STD_LOGIC ;
    signal data_out : STD_LOGIC;


begin

    UUT : simple_latch
    Port map(
        data_in => data_in,
       -- R => R,
        CLOCK => CLOCK,
        data_out => data_out
        --QBAR => QBAR
    );
     data_in <= not data_in after 10 ns;
    --R <= not R after 20 ns;
    CLOCK <= not CLOCK after 20 ns;
   -- D <= not D after 80 ns;

end Behavioral;
```

Simulation output



JK flip-flop

- JK flip-flop accounts the illegal condition in SR flip flop.

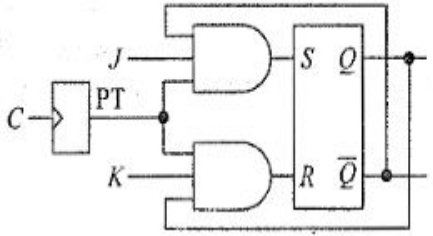
- When J and K are both low, both AND gates are disabled. Therefore, clock pulses have no effect. So, Q retains its last value.

- When J is low and K is high, the upper gate is disabled, so there's no way to set the flip-flop.
- It means that the next PT of the clock resets the flip-flop (unless Q is already reset).

- When J and K are both high, it's possible to set or reset the flip-flop.
- If Q is high, the lower gate passes a RESET pulse on the next PT. On the other hand, when Q is low, the upper gate passes a SET pulse on the next PT.

- Either way, Q changes to the complement of the last state.

Truth table and digital circuit for JK FF



C	J	K	Q_{n+1}	Action
↑	0	0	Q_n (last state)	No change
↑	0	1	0	RESET
↑	1	0	1	SET
↑	1	1	$\overline{Q}_n$ (toggle)	Toggle

http://soaneemrana.com/onewebmedia/DIGITAL%20PRINCIPLES%20AND%20APPLICATION%20BY%20LEACH%20&%20MALVINO.pdf

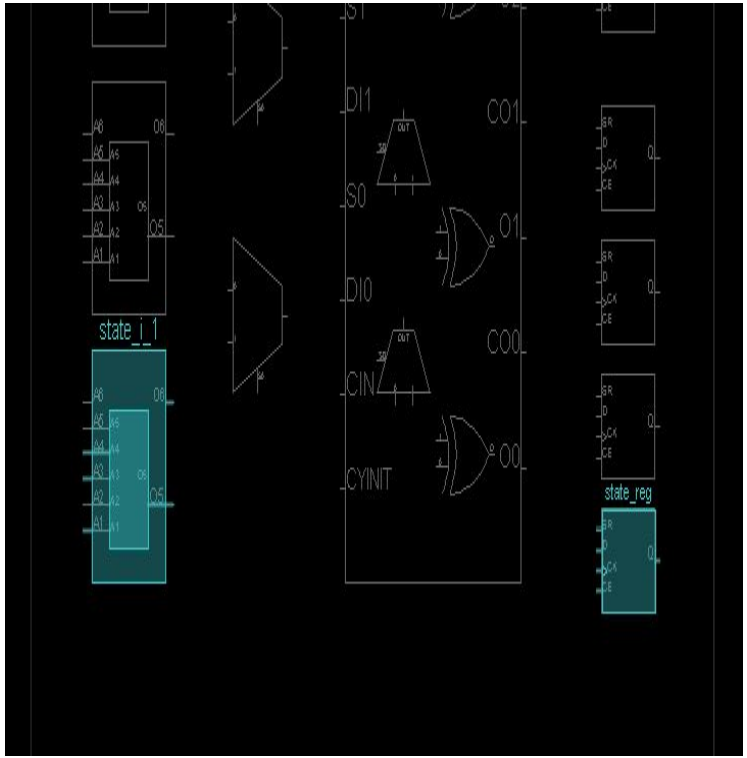
JK Flip Flop

VHDL code

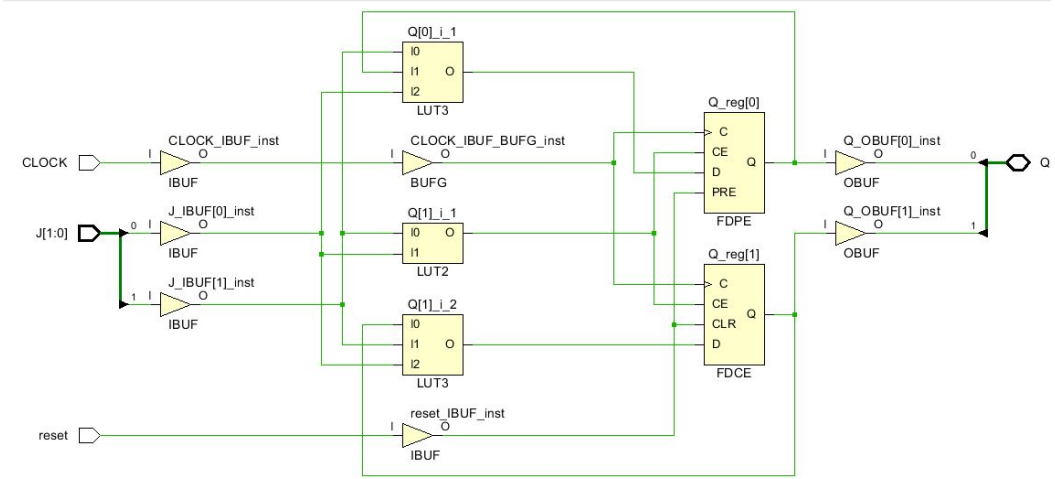
```
entity JK_flipflop is
    Port ( J : in STD_LOGIC_VECTOR (1 downto 0);
           Q : inout STD_LOGIC_VECTOR (1 downto 0);

           CLOCK, reset : in STD_LOGIC);
end JK_flipflop;

architecture Behavioral of JK_flipflop is

begin
process(CLOCK)
       --  variable state : STD_LOGIC;
         begin
             if(reset ='1') then
              Q <= "01";
             elsif (CLOCK = '1' and CLOCK'EVENT) then
                   if ( J ="00") then
                     -- state:= state;
                     Q <= Q;
                   elsif( J = "11") then
                     -- state:= not state;
                     Q <= not Q;
                   elsif ( J = "01") then
                    Q  <= "01";
                   else
                      Q<= "10";
                   end if;
              end if;

          end process;
```

JK Flip Flop

Implemented design of source code in fpga



Schematic representation of JK FF



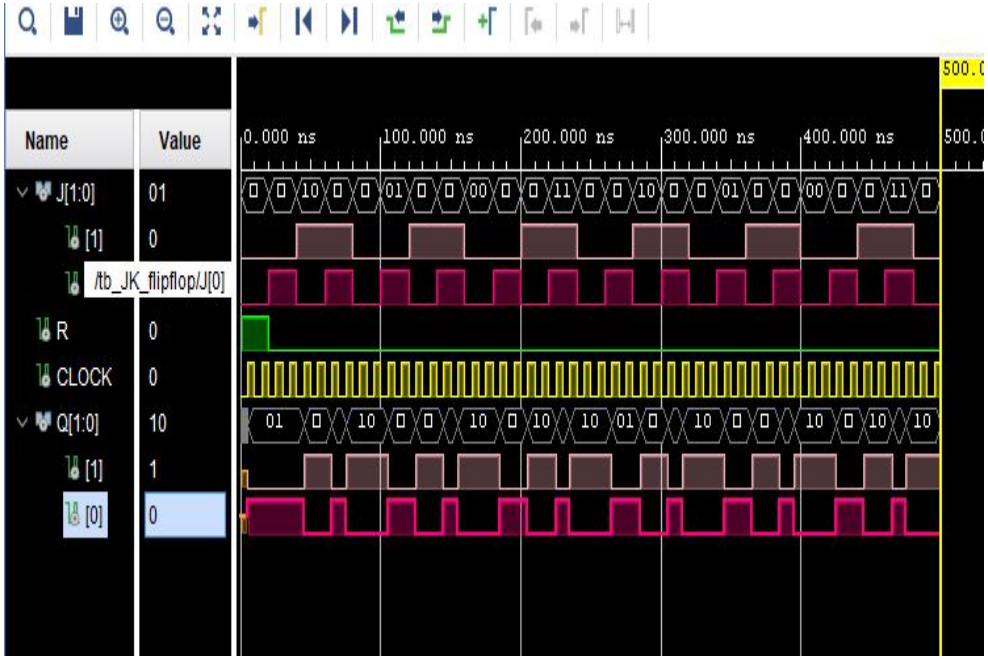
Test bench code and simulation output for JK flip-flop

Test bench code

```
entity tb_JK_flipflop is
--  Port ( );
end tb_JK_flipflop;

architecture Behavioral of tb_JK_flipflop is

component JK_flipflop is
    Port (
        J : in STD_LOGIC_vector (1 downto 0);
        reset : in STD_LOGIC;
        CLOCK : in STD_LOGIC;
        q : inout STD_LOGIC_vector (1 downto 0)
        --QBAR : out STD_LOGIC
    );
end component JK_flipflop;

    signal J : STD_LOGIC_vector ( 1 downto 0) := "00"
    signal R : STD_LOGIC := '0';
    signal CLOCK : STD_LOGIC := '0';
    signal Q : STD_LOGIC_vector ( 1 downto 0) ;
    --signal QBAR : STD_LOGIC;
    constant clk_period : time := 10 ns;

begin

    UUT : JK_flipflop
    Port map(
        J => J,
        reset => R,
        CLOCK => CLOCK,
        q=> Q
        --QBAR => QBAR
    );
    clk_process :process
   begin
   clock <= '0';
   wait for clk_period/2;
   clock <= '1';
   wait for clk_period/2;
   end process;

     stim_proc: process
```

```
begin

    UUT : JK_flipflop
    Port map(
        J => J,
        reset => R,
        CLOCK => CLOCK,
        q=> Q
        --QBAR => QBAR
    );
    clk_process :process
   begin
   clock <= '0';
   wait for clk_period/2;
   clock <= '1';
   wait for clk_period/2;
   end process;

     stim_proc: process
   begin

   R <= '1';
   wait for 20 ns;

   R <= '0';
    wait for 50 ns;

   R <= '0';
   wait;
   end process;

      J(0) <= not J(0) after 20 ns;
      J(1) <= not J(1) after 40ns;
      end behavioral;
```

Simulated output



Exercise: 5 - **Multiplexers**

- **2-to-1 Multiplexer**
  - Overview of 2-to-1 Multiplexer
  - VHDL code, Schematic representation
  - Test bench and simulated output

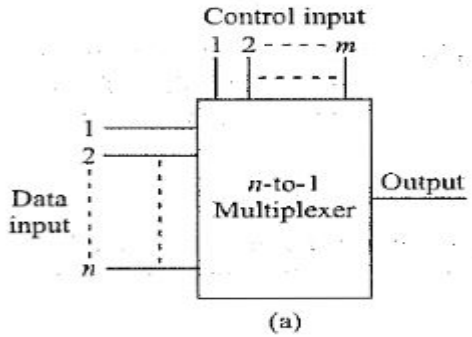
- **4-to-1 Multiplexer**
  - Overview of 4-to-1 Multiplexer
  - VHDL code, Schematic representation
  - Test bench and simulated output

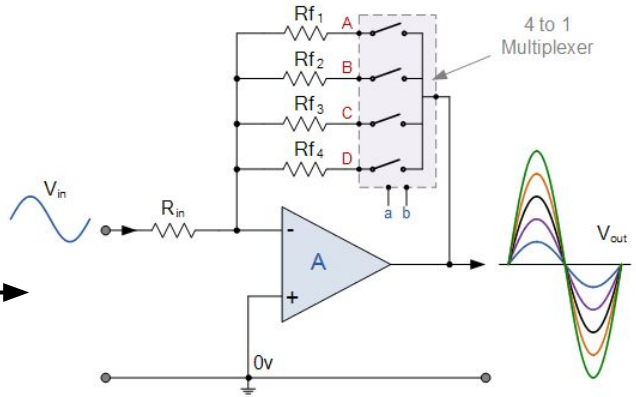
- **8-to-1 Multiplexer using two 4-to-1 Multiplexer**
  - Overview of 8-to-1 Multiplexer
  - VHDL code, Schematic representation
  - Test bench and simulated output

- **16-to-1 Multiplexer using two 8-to-1 Multiplexer**
  - Overview of 2-to-1 Multiplexer
  - VHDL code, Schematic representation
  - Test bench and simulated output

Multiplexer

- Sometimes called as “Data Selector”.
- A circuit with many inputs but with only one output.
- For m control inputs, we can have maximum **2^m** inputs.
- According to the binary condition of control inputs, either “HIGH” or “LOW” the appropriate data input is connected directly to the output.
- It reduces the number of logic gates required in a circuit design or when a single data line or data bus is required to carry two or more different digital signals.
- One use is in **Digitally Adjustable Amplifier Gain.**

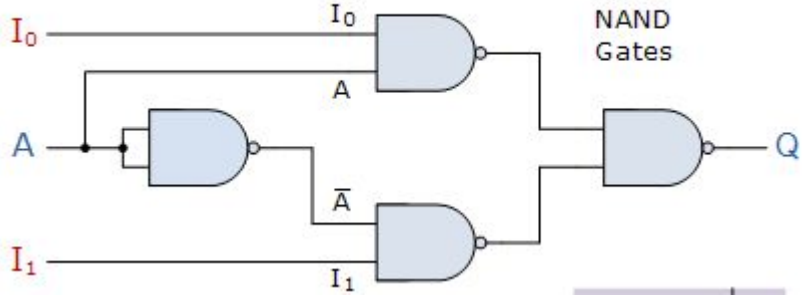




https://www.electronics-tutorials.ws/combination/comb_2.html

2-to-1 Multiplexer

- When the data select input, A is LOW at logic 0, input I_1 passes its data through the NAND gate multiplexer circuit to the output.
- While input I_0 is blocked. When the data select A is HIGH at logic 1, the reverse happens and now input I_0 passes data to the output Q while input I_1 is blocked.
- In 2-to-1, we select one input out of two inputs at a time.



$$Q = \overline{A.I_0}.I_1 + \overline{A}.I_0.I_1 + A.I_0.\overline{I_1} + A.I_0.I_1$$

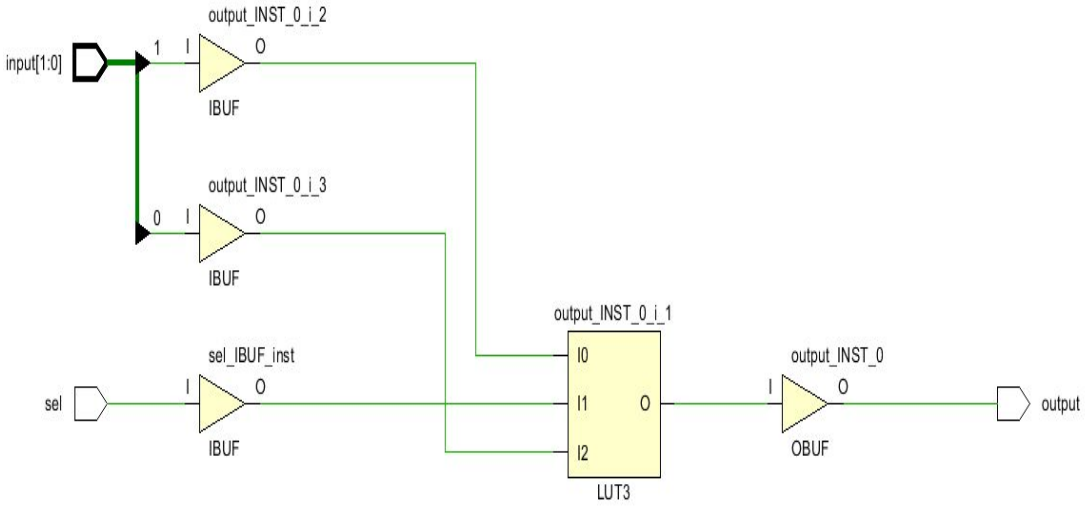
http://soaneemrana.com/onewebmedia/DIGITAL%20PRINCIPLES%20AND%20APPLICATION%20BY%20LEACH%20&%20MALVINO.pdf

2-to-1 Multiplexer

VHDL code

```
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity multiplexer_2bit is
    Port ( input : in STD_LOGIC_VECTOR (1 downto 0);
           output : out STD_LOGIC;
           sel : in STD_LOGIC);
end multiplexer_2bit;

architecture Behavioral of multiplexer_2bit is

begin
process( sel, input)
begin
     if (sel = '0') then
     output <= input(0);
     elsif ( sel = '1') then
     output <= input(1);
     else
     output <='0';
     end if;
 end process;

end Behavioral;
```

Synthesized design



Test bench and simulated output for 2-to-1 Multiplexer

Test bench code

```
end tb_multiplexer_2bit;

architecture Behavioral of tb_multiplexer_2bit is

component multiplexer_2bit is ---- declared here which
port (
 input : in STD_LOGIC_VECTOR ( 1 downto 0);
 output : out STD_LOGIC;
 sel : in STD_LOGIC
 );
 end component multiplexer_2bit;

 signal input : STD_LOGIC_VECTOR ( 1 downto 0) := "00"
 signal output :  STD_LOGIC;
 signal sel :  STD_LOGIC := '0';
 --constant input_period : time:= 10ns;

begin

UUT : multiplexer_2bit  ---- name of source vhdl file
port map(  ---- mapping of source entity to test bench
   input => input,
   output => output,
   sel => sel
   );

  sel <= not sel after 20ns;
  input(0) <= not input(0) after 10ns;
  input(1) <= not input(1) after 40ns;


end Behavioral;
```

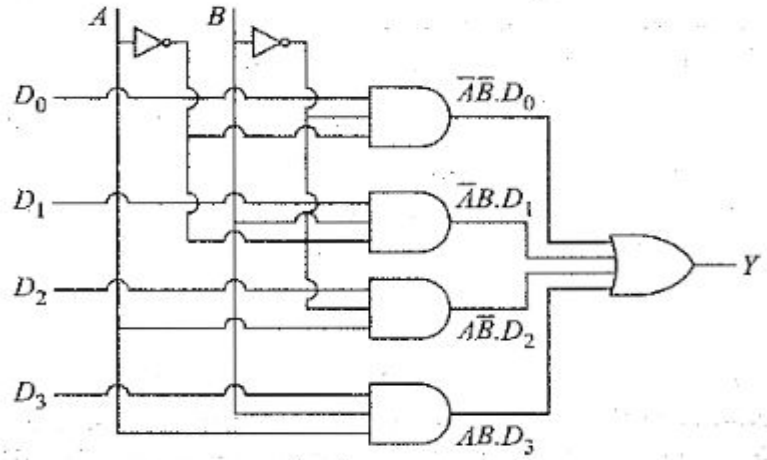
Simulated output

4-to-1 Multiplexer

- Depending on control inputs A, Bone of the four inputs Do to D3 is steered to output Y.

- For AB = 00, the first AND gate to which D0 is connected remains active and equal to Do and all other AND gate are inactive with output held at logic 0.

Logic circuit



Truth table

A	B	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

One example

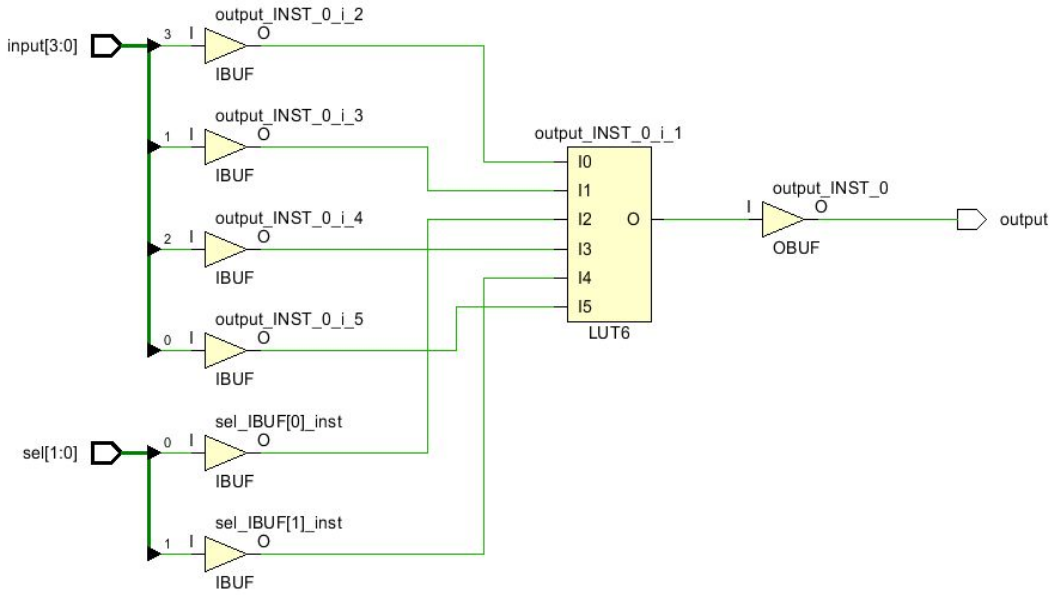
$$Y = A'B'.D_0 + A'B.D_1 + AB'.D_2 + AB.D_3$$
$$Y = 0'0'.D_0 + 0'.0.D_1 + 0.0'.D_2 + 0.0.D_3$$
$$Y = 1.1.D_0 + 1.0.D_1 + 0.1.D_2 + 0.0.D_3$$
$$Y = D_0$$

4-to-1 Multiplexer

VHDL code

```
--use UNISIM.VComponents.all;

entity multiplexer_4_to_1 is
    Port ( input : in STD_LOGIC_VECTOR (3 downto 0);
           sel : in STD_LOGIC_VECTOR (1 downto 0);
           output : out STD_LOGIC);
end multiplexer_4_to_1;

architecture Behavioral of multiplexer_4_to_1 is

begin
slection : process( sel,input) is
           begin
                if ( sel = "00") then
                output <= input(0);
                elsif (sel = "01") then
                output <= input(1);
                elsif ( sel = "10") then
                output <= input(2);
                elsif( sel = "11") then
                output <= input(3);

                else
                output <= '0';
                end if;
            end process slection;

end Behavioral;
```

Synthesized design

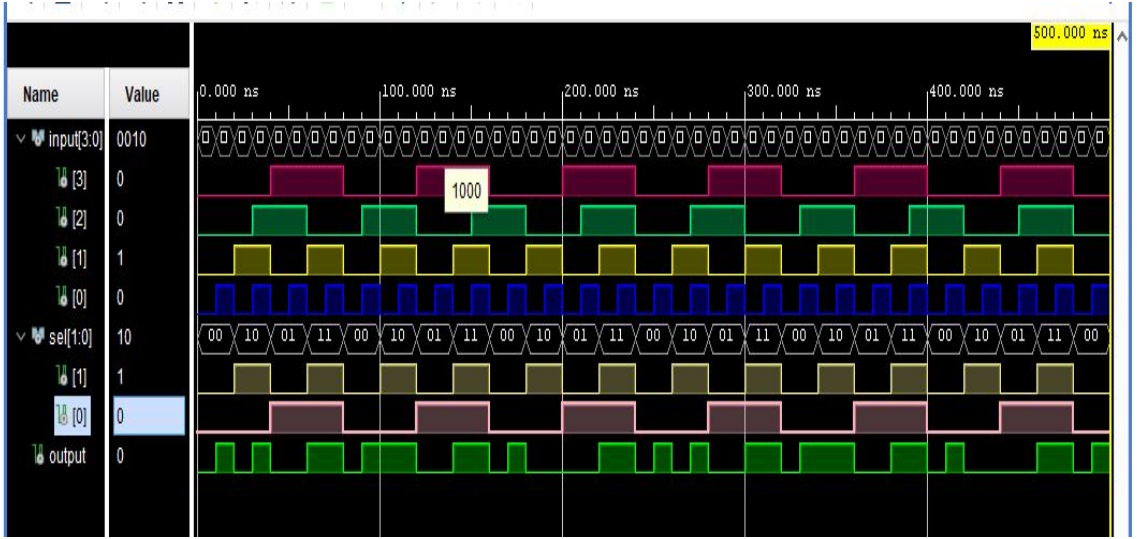


Test bench and simulated output for 4-to-1 Multiplexer

Test bench code

```
entity tb_multiplexer_4_to_1 is
--  Port ( );
end tb_multiplexer_4_to_1;

architecture Behavioral of tb_multiplexer_4_to_1 is

component multiplexer_4_to_1 is ---- declared here which
port (
 input : in STD_LOGIC_VECTOR ( 3 downto 0);
 output : out STD_LOGIC;
 sel : in STD_LOGIC_vector (1 downto 0)
 );
 end component  multiplexer_4_to_1;

 signal input : STD_LOGIC_VECTOR ( 3 downto 0) := "0000";
 signal output :  STD_LOGIC;
 signal sel :  STD_LOGIC_VECTOR(1 downto 0) := "00";

begin

UUT : multiplexer_4_to_1
 port map(

  input => input,
   output => output,
   sel => sel
   );

  sel(0)<= not sel(0) after 40ns;
  sel(1) <= not sel(1) after 20ns;
  input(0) <= not input(0) after 10ns;
  input(1)<= not input(1) after 20ns;
    input(2)<= not input(2) after 30ns;
  input(3)<= not input(3) after 40ns;


end Behavioral;
```

Simulated output



8-to-1 Multiplexer

- We can make 8-to-1 MUX using two 4-to-1 MUX as inputs to a 2-to-1 MUX.
-

Next steps

1. Synchronized multiplexers
2. De - multiplexers
3. Counters (...4 bit, 8 bit)
    a. Synchronised counter

4. Registers using flip- flops
    a. Shift registers

5. Coder & Decoder

Thank you for all the help - suggestions and comments will be very helpful !